SAÉ 2.03 : Installation de services réseaux (Question Réponse)

Table des matières

1. Semaine 7 : Questions et Réponses techniques 1
2. Semaine 8 : Comparatif Markdown vs AsciiDoc 5
3. Semaine 10 : Questions et Réponses techniques (Git) 9
4. Semaine 11 : Questions et Réponses techniques (Forgejo) 13

1. Semaine 7 : Questions et Réponses techniques

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

Navigation rapide : Pour voir notre démarche d'installation détaillée, veuillez ouvrir le fichier **SAE203_Tutoriel.pdf** au chapitre **Semaine 7**.

Question 1 : Que signifie "64-bit" ?
Cela désigne l'architecture du processeur capable de traiter des données par blocs de 64 bits. C'est l'architecture la plus courante pour les systèmes modernes.
Source : Wikipédia - Processeur 64 bits

Question 2 : Quelle est la configuration réseau de la carte réseau virtuelle émulée par défaut ?
Par défaut, VirtualBox émule un adaptateur réseau ethernet `Intel PRO/1000 MT Desktop` avec un câble virtuel connecté. `PCNet FAST III` est l'adaptateur par défaut pour les anciennes machines, mais les Debian utilisent le nouvel adaptateur.
Source : Manuel officiel VirtualBox

Question 3 : Pourquoi allouer 2048 Mo de RAM à la VM ? Que se passerait-il avec seulement 512 Mo ?
2048 Mo permettent de faire tourner confortablement l'installation ainsi que l'environnement graphique MATE. Avec seulement 512 Mo, le système utiliserait massivement la partition d'échange (Swap), rendant la machine extrêmement lente, et l'interface graphique pourrait même refuser de se lancer par manque de mémoire.
Source : Documentation Debian - Configuration matérielle

Question 4 : Quel est le mode réseau par défaut dans VirtualBox ?
Le mode NAT. Il isole la VM tout en lui donnant accès à Internet via la connexion de la machine hôte.
Source : Documentation VirtualBox - Mode NAT

Question 5 : Quelle est l'adresse IP de votre VM et comment l'obtenir ?
La commande utilisée est `ip address`. L'adresse IP locale attribuée par le NAT est `10.0.2.15`.
Source : Page de manuel de la commande IP

Question 6 : Quelle est l'adresse de la passerelle de la VM ?
La commande utilisée est `ip route`. La passerelle par défaut est `10.0.2.2`. Elle correspond au routeur virtuel créé par VirtualBox pour le mode NAT.
Source : Page de manuel de la commande IP

Question 7 : Qu'est-ce qu'un fichier iso bootable ?
C'est une image disque (une copie exacte d'un CD/DVD) contenant les fichiers nécessaires pour amorcer (booter) un ordinateur et installer un système d'exploitation.
Source : FAQ officielle des images CD Debian

Question 8 : Que sont MATE et GNOME ?
Ce sont des Environnements de Bureau (Desktop Environments) pour Linux. Ils fournissent l'interface graphique (fenêtres, icônes, barres des tâches). GNOME est plus lourd et moderne, MATE est un dérivé plus léger.
Source : Wiki Debian - Environnements de bureau

Question 9 : Qu'est-ce qu'un serveur mandataire (proxy) ?
C'est un serveur qui agit comme intermédiaire entre les requêtes d'un client (notre VM) et un autre serveur (Internet). Il peut filtrer, mettre en cache ou sécuriser les connexions.
Source : Wikipédia - Serveur mandataire

Question 10 : Qu'est-ce qu'un serveur web ?
Un logiciel (comme Apache ou Nginx) qui répond aux requêtes HTTP/HTTPS pour distribuer des pages web à des clients (navigateurs).
Source : Wiki Debian - Serveur Web

Question 11 : Quel logiciel de serveur SSH a été installé ? Comment vérifier qu'il est bien démarré ?
Le paquet installé est `openssh-server`. On vérifie son état avec la commande `systemctl status ssh`.

Question 12 : Pourquoi la connexion SSH (ssh user@10.0.2.15) échoue-t-elle avec un "Timeout" ?
Parce que le mode réseau NAT de VirtualBox crée un réseau séparé de celui de l'hôte. Il faut configurer une redirection de ports (Port Forwarding) dans VirtualBox pour que cela fonctionne.
Source : Documentation VirtualBox - Redirection de ports

Question 13 : Comment peut-on savoir à quels groupes appartient l'utilisateur user ?
Pour savoir à quels groupes appartient l'utilisateur `user`, on utilise la commande `groups user` ou `id user`
Source : Page de manuel groups
Source : Page de manuel id.

Question 14 : Quelle est la différence entre `su` et `sudo` ?
`su` permet de changer d'utilisateur (souvent pour devenir root) et requiert le mot de passe de l'utilisateur cible.
`sudo` permet d'exécuter une seule commande avec les privilèges d'un autre utilisateur (root en général), mais nécessite le mot de passe de l'utilisateur courant.
Source : Page de manuel sudo
Source : Page de manuel su

Question 15 : Quelle est la version du noyau Linux utilisé par votre VM ?
`uname -r` : `6.12.73+deb13-amd64`
Source : Page de manuel uname

Question 16 : Quelle est l'utilité des suppléments invités (Guest Additions) ?
Ils optimisent la VM. Ils permettent notamment le redimensionnement dynamique de la fenêtre (ajustement de la résolution) et activent le presse-papiers partagé entre la machine hôte et la VM.
Source : Wiki Debian - Additions Invité VirtualBox

Question 17 : À quoi sert la commande mount ?
`mount` sert en général à attacher un système de fichiers (disque, clé USB) à l'arborescence du système (point de montage). Dans notre cas, elle a servi à rendre accessible le contenu du lecteur CD-ROM virtuel pour y exécuter le script d'installation.
Source : Page de manuel mount

Question 18 : Qu'est-ce que le Projet Debian ?
C'est une association d'individus qui ont pour cause commune de créer un système d'exploitation libre.
Source : Site officiel du projet Debian

Question 19 : D'où vient le nom Debian ?
Le nom "Debian" a été imaginé par son créateur, Ian Murdock, en combinant le prénom de sa petite amie de l'époque (Debra) et le sien (Ian).
Source : Historique du projet Debian

Question 20 : Il existe 3 durées de prise en charge (support) de ces versions : la durée minimale, la durée en support long terme (LTS) et la durée en support long terme étendue (ELTS). Quelle sont les durées de ces prises en charge ?

- *Minimale (Standard)* : Environ 3 ans (1 an après la sortie de la version stable suivante).
- *LTS (Long Term Support)* : 5 ans au total. Debian LTS
- *ELTS (Extended LTS)* : Jusqu'à 10 ans (géré par une organisation tierce, Freexian). Debian ELTS

Question 21 : Pendant combien de temps les mises à jour de sécurité seront-elles fournies ?
L'équipe de sécurité Debian fournit un support pour la version stable pendant environ 1 an après la sortie d'une nouvelle version stable.
Source : Debian Release Lifecycle

Question 22 : Quelles sont les différentes versions de Debian activement maintenues ?
Il y en a au minimum 3. Leurs noms génériques sont : *Stable*, *Testing* (en test), et *Unstable* (instable, surnommée *Sid*). On peut aussi compter *Oldstable*.
Source : Debian Release Lifecycle

Question 23 : D'où proviennent les noms de code des versions Debian ?
Tous les noms de code des versions Debian proviennent des personnages des films d'animation *Toy Story* (ex. : Bullseye, Bookworm, Trixie).
Source : Toy Story - Debian Wiki

Question 24 : L'un des atouts de Debian fut le nombre d'architecture (¬ processeurs) officiellement prises en charge. Combien et lesquelles sont prises en charge par la version Trixie ?
La version Trixie de Debian supporte: *amd64*, *aarch64*, *armhf*, *powerpc*, *riscv64* et *system z*.
Source : Debian Trixie Release Information

Question 25 : Quel a été le premier nom de code utilisé ? Quand a-t-il été annoncé ? Quelle était le numéro de version de cette distribution ?
`Buzz` (Debian 1.1), annoncé le 17 juin 1996.
Source : Debian Releases

Question 26 : Quel est le dernier nom de code annoncé à ce jour ? Quand a-t-il été annoncé ? Quelle est la version de cette distribution ?
La version 14 est `Forky` et la version 15 sera `Duke`.
Source : Annonce de Forky - Debian Mailing List

Question 27 : Quelle est la différence entre `pkgsel/include` et `tasksel` dans un preseed ?
`d-i pkgsel/include` permet de spécifier une liste de **paquets individuels** précis à installer (ex. : `git`, `curl`).
`tasksel tasksel/first` installe des **tâches entières** qui regroupent des centaines de paquets (ex. : `mate-desktop` ou `web-server`)
Source : Wiki Debian - Tasksel

Question 28 : Le fichier preseed pose-t-il un problème de sécurité ?

Oui, le fichier preseed contient les mots de passe de `root` et de `user` en clair. En production, il faut protéger ce fichier en le stockant sur un serveur web interne sécurisé (HTTPS avec authentification), en restreignant drastiquement les droits de lecture du fichier physique ou en écrivant le hash uniquement
Source : Manuel Debian - Sécurité de la pré-configuration

2. Semaine 8 : Comparatif Markdown vs AsciiDoc

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

Dans le cadre de notre projet, nous avons comparé deux langages de balisage léger très populaires : le Markdown et l'AsciiDoc. Voici un grand tableau comparatif détaillant leurs concepts, origines et syntaxes.

Description	Langage de balisage léger très populaire, conçu pour être facile à lire et à écrire. Très orienté pour le web et les plateformes collaboratives (ex: GitHub).	Langage de balisage léger plus riche et standardisé, conçu initialement pour la rédaction d'articles, de livres techniques et de documentations complexes.
Lien vers la documentation officielle	Site officiel Markdown	Documentation Asciidoctor
Origine (Date et Créateurs)	Créé en **2004** par John Gruber (avec l'aide d'Aaron Swartz).	Créé en **2002** par Stuart Rackham.

Comment convertir (sans extension) vers HTML et PDF	Utilisation classique avec la commande **Pandoc** :
	`pandoc fichier.md -o fichier.html`
	`pandoc fichier.md -o fichier.pdf`
		Utilisation avec la suite **Asciidoctor** :
		`asciidoctor fichier.adoc`
		`asciidoctor-pdf fichier.adoc`

Écrire en gras	Pour écrire en gras, il faut utiliser deux astérisques de chaque côté du mot : ****mot****	
Écrire en italique	Pour écrire en italique, il faut utiliser un tiret bas : *_mot_*	
Écrire en gras ET italique	Pour combiner les deux, on cumule les symboles : **_***mot***_**	
Texte souligné	Le Markdown natif ne possède pas de syntaxe pour souligner. Il faut utiliser les balises HTML : `<u>` mot `</u>`	AsciiDoc gère le soulignement grâce aux rôles (les espaces autour sont importants) : [.underline]# mot #
Texte surligné	Cela dépend des extensions (parfois `==` mot `==`), ou via la balise HTML : `<mark>`mot`</mark>`	AsciiDoc utilise simplement le symbole dièse de chaque côté (les espaces autour sont importants) : # mot #
Texte barré	On utilise très souvent deux tildes : ~~ ~~mot~~ ~~	AsciiDoc utilise les rôles : [.line-through]# ~~mot~~ #

<table>
<tr><td></td><td></td><td></td></tr>
<tr><td>Faire des listes à puces (avec sous-liste)</td><td>On utilise des astérisques, avec une indentation (espaces) pour le sous-niveau :

<pre>* Élément principal 1
* Élément principal 2
  * Sous-élément</pre></td><td>On utilise des astérisques, et on en ajoute un supplémentaire par niveau de profondeur :

<pre>* Élément principal 1
* Élément principal 2
** Sous-élément</pre></td></tr>
<tr><td>Faire des listes numérotées</td><td>On utilise des nombres suivis d’un point :

<pre>1. Première étape
2. Deuxième étape</pre></td><td>Asciidoc gère la numérotation automatiquement avec des simples points :

<pre>. Première étape
. Deuxième étape</pre></td></tr>
<tr><td>Créer un tableau simple</td><td>On le dessine avec des barres verticales (<code>|</code>) et des tirets pour séparer l’en-tête : <pre>| Colonne 1	Colonne 2
Valeur 1	Valeur 2 |</pre></td><td>On déclare le nombre de colonnes puis on liste les cellules les unes après les autres : <pre>[cols="1,1", options="header"]
!===
! Colonne 1 ! Colonne 2
! Valeur 1 ! Valeur 2
!===</pre>
Table 1. Résultat :
<table><tr><td></td><td></td></tr><tr><td>Valeur 1</td><td>Valeur 2</td></tr></table></td></tr>
<tr><td>Différents types de tableaux (tailles et fusions)</td><td>Très limité. Le Markdown classique ne gère ni les tailles de colonnes précises, ni la fusion de cellules (il faut coder en HTML).</td><td>Très puissant.
On ajuste la taille des colonnes avec <code>cols</code>. On fusionne des colonnes avec <code>2+</code> :

<pre>[cols="1,3"]
!===
2+! Titre fusionné sur 2 colonnes
! Case 1 petite ! Case 2 très large
!===</pre>
Table 2. Résultat :
<table><tr><td colspan="2">Titre fusionné sur 2 colonnes</td></tr><tr><td>Case 1 petite</td><td>Case 2 très large</td></tr></table></td></tr>
</table>

Insérer un lien hypertexte	Le texte est entre crochets, l'URL entre parenthèses : [Debian](https://debian.org)	L'URL est au début, le texte est entre crochets à la fin : https:/¬/debian.org[Debian]
Insérer une image	Un point d'exclamation, le texte alternatif, puis le lien : ![titreAlternatif](image/imageMarkdown.png)	La macro image, suivie des attributs (taille, alignement) : `image::image/imageAdoc.png[titreAlternatif, width=50%]`

Blocs d'informations (Admonitions : Notes, Avertissements…)	Le Markdown natif ne le gère pas. (Il faut utiliser des extensions spécifiques de Pandoc).	Intégré nativement avec des icônes.
		`[NOTE]`
		`====`
		`Ceci est une note.`
		`(Les autres types sont TIP, WARNING, CAUTION, IMPORTANT)`
		`====`
		Résultat :
		Ceci est une note. (Les autres types sont TIP, WARNING, CAUTION, IMPORTANT)

Blocs de code avec coloration syntaxique	Trois accents graves suivis du langage :
	``bash
	echo "Bonjour"
	``
		Directive source suivie du langage, puis délimiteurs `----` (on utilise `....` pour imbriquer l'exemple sans casser le tableau) :
		`[source,bash]`
		`....`
		`echo "Bonjour"`
		`....`

Citations (Quotes)	On utilise le chevron vers la droite `>` en début de ligne :
	`> Ceci est une citation de texte.`
	
	AsciiDoc offre aussi des blocs avancés pour les citations :
	`[quote, Auteur, Source]`
	`____`
	`Ceci est une citation avancée en AsciiDoc.`
	`____`
	Résultat :
	> Ceci est une citation avancée en AsciiDoc.
	> — Auteur, Source	

Notes de bas de page	On doit créer une ancre et son contenu :
	`Ceci est mon texte[^1].`
	`[^1]: Et voici la note de bas de page.`
		Tout se fait sur la même ligne avec la balise footnote :
		`Ceci est mon texte footnote:[Et voici la note directement intégrée ici].`

Retour d'expérience : Installation locale d'Asciidoctor

Lors de la préparation de notre environnement de travail en local (sur les machines de l'IUT), nous avons rencontré une difficulté majeure : l'impossibilité d'installer Asciidoctor de manière globale par manque de droits d'administration (accès `root` bloqué).

La solution a été de forcer l'installation des paquets Ruby directement dans notre espace utilisateur grâce à l'option `--user-install` :

```
gem install --user-install asciidoctor asciidoctor-pdf rouge
```

Il a ensuite fallu ajouter le chemin des binaires Ruby au `PATH` de notre environnement utilisateur pour pouvoir exécuter les commandes de compilation.

3. Semaine 10 : Questions et Réponses techniques (Git)

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

Navigation rapide : Pour voir nos manipulations et captures d'écran sur Git, veuillez ouvrir le fichier **SAE203_Tutoriel.pdf** au chapitre **Semaine 10**. ./SAE203_Tutoriel.html#tuto-semaine10[test]

Question 1 : Quelle est la différence entre Git et les logiciels comme GitHub/GitLab/Forgejo ?
Git est l'outil de contrôle de version local en ligne de commande, alors que GitHub/GitLab/Forgejo sont des forges logicielles collaboratives (des plateformes web) permettant d'héberger et de gérer les dépôts Git en ligne.
Source : Documentation officielle Git - Dépôts distants

Question 2 : Qu'est-ce qu'un dépôt Git ? Où sont stockées les données d'un dépôt local ?
Un dépôt Git, souvent appelé "repo", est un emplacement de stockage pour les fichiers d'un projet et l'ensemble de l'historique de leurs modifications. Ces données sont stockées de façon masquée dans le dossier local `.git`.
Source : Documentation Git - Le répertoire Git

Question 3 : Quelle est la différence entre un commit, une branche et un tag ?
Un **commit** est un instantané contenant les différences de chaque fichier par rapport au commit précédent.
Une **branche** est une ligne de développement indépendante (une suite de commits).
Un **tag** est une étiquette figée collée sur un commit précis (souvent utilisée pour marquer les versions logicielles).
Source : Documentation Git - Les Branches

Question 4 : Qu'est-ce qu'un dépôt distant (remote) ? Comment lister les remotes configurés ?
Un dépôt distant est un dépôt accessible par le réseau (via Internet ou un réseau local d'entreprise). La commande `git remote -v` permet de lister ceux qui sont configurés sur le projet.
Source : Documentation Git - Commandes Remote

Question 5 : Quelle est la différence entre git pull et git fetch ?
`git fetch` : récupère les données du dépôt distant vers le dépôt local (sans modifier vos fichiers de travail en cours).
`git pull` : effectue un **fetch** puis fusionne directement ces nouvelles données avec la branche locale actuelle.
Source : Documentation Git - Fetch vs Pull

Question 6 : Quels sont les protocoles supportés par Git pour accéder à un dépôt ?
Protocole Local (accès au système de fichiers sans port)
Protocole Git
Protocole HTTP/HTTPS (mode intelligent et idiot) [1]
Protocole SSH.
Source : Livre Git - Les Protocoles

Question 7 : Sur quels ports réseau fonctionnent ces protocoles ?
Le protocole Git fonctionne sur le port `9418`
HTTP sur le port `80`
HTTPS sur le port `443`
SSH sur le port `22`.
Source : Documentation Git - Les protocoles réseau

Question 8 : Comment configurer l'authentification SSH pour Git ?
Il faut générer une paire de clés SSH sur l'ordinateur avec la commande `ssh-keygen -t ed25519 -C "exemple@univ-lille.fr"`, puis copier le contenu de la clé publique générée pour l'ajouter dans les paramètres de sécurité du compte sur la forge logicielle.
Source : Documentation Git - Génération clé SSH

Question 9 : Créez un nouveau dépôt Git local. Quelle commande utilisez-vous ? Que se passe-t-il ?
La commande `git init` crée un sous-dossier `.git` qui initialise la structure d'un dépôt vide dans le répertoire actuel.
Source : Documentation Git - git init

Question 10 : Utilisez git log pour visualiser l'historique.
Le résultat est une liste des commits du plus récent au plus ancien avec leurs identifiants (be99f4166…), l'auteur, la date et le message de commit.
Source : Documentation Git - git log

Question 11 : Créez une nouvelle branche, faites des modifications, puis fusionnez (merge) cette branche avec la principale. Expliquez.
Une branche est une chaine de commits parallèle aux autres branches. La fusion (merge) permet de combiner les changements d'une branche dans une autre en rajoutant les commits de la branche source vers la branche cible.
Source : Documentation Git - git merge

Question 12 : Qu'est-ce qu'un conflit de fusion ? Provoquez-en un et montrez comment le résoudre.
Un conflit survient lorsque deux branches modifient le même fichier de façon différente. Lors de la fusion, Git met le processus en pause et demande une résolution manuelle (choisir le code à garder et supprimer les marqueurs de conflit). Pour résoudre le conflit, on modifie le fichier puis on le `git add` pour le marquer comme résolu, avant de faire un commit de fusion.
Source : Documentation Git - Résolution de conflits

Question 13 : Utilisez git diff pour comparer deux commits. Expliquez la sortie.

```
diff --git a/texte b/texte
index 9d600a1..7255def 100644
--- a/texte
+++ b/texte
@@ -1 +1 @@
-dans une branche
+conflit 1
```

Les premières lignes indiquent les fichiers comparés (`a/texte` et `b/texte`) et les méta-données de la comparaison (`index 9d600a1..7255def`). Les lignes précédées d'un `-` en rouge sont des suppressions et celles avec un `+` en vert sont des ajouts. Les autres lignes affichées servent de contexte autour du code modifié.

Question 14 : Qu'est-ce que le fichier .gitignore ? Créez-en un pour ignorer les fichiers Java (.class, .jar).
C'est un fichier texte permettant de lister les motifs des fichiers/dossiers qui ne doivent pas être suivis et versionnés par Git.
Source : Documentation Git - gitignore

Question 15 : Création d'un dépôt privé partagé sur GitLab et clonage par chaque membre.
Il faut faire `git clone git@gitlab-ssh.univ-lille.fr:notthedreamteam/exemple-sae203.git`.
Source : Documentation GitLab - Clonage d'un dépôt

Question 16 : Chaque membre crée une branche, ajoute un fichier, et pousse (push) sa branche.
Commandes utilisées :

```
git switch --create emilien
git add
git commit -m "ajout du fichier"
git push --set-upstream origin emilien
```

Source : Documentation Git - Les branches

Question 17 : Fusion de toutes les branches dans la principale. Documentez les éventuels conflits.
On a fait des merge requests dans gitlab pour fusionner les branches. Nous n'avons pas eu de conflits car nous n'avons pas modifié les mêmes fichiers.
Source : Documentation GitLab - Merge Requests

Question 18 : Qu'est-ce qu'une pull request (ou merge request) ? À quoi sert-elle ?
Une pull/merge request sert à demander la fusion d'une branche source vers une branche destination tout en permettant aux autres membres de vérifier le code avant validation.
Source : Documentation GitLab - Merge Requests

Question 19 : Testez la commande git blame. À quoi sert-elle ?
La commande `git blame fichier` permet d'afficher le fichier ligne par ligne en indiquant pour chaque ligne l'auteur, la date et le hash du commit qui l'a modifiée pour la dernière fois. C'est très utile pour retrouver l'origine d'un bout de code ou d'un bug.
Source : Documentation officielle Git - git blame

Question 20 : Qu'est-ce que le logiciel gitk ? Comment se lance-t-il ?
C'est un visualiseur graphique pour les dépôts Git. Il permet d'explorer l'historique des commits, l'arborescence des branches, et de lire les diffs plus confortablement. Il se lance simplement en tapant la commande `gitk` dans le terminal depuis un dépôt Git.
Source : Documentation officielle Git - gitk

Question 21 : Qu'est-ce que le logiciel git-gui ? Comment se lance-t-il ?
C'est une interface graphique pour la création de commits. Contrairement à gitk, il permet de sélectionner (stage) des fichiers, d'écrire le message de commit et de valider. Il se lance en tapant `git gui` dans le terminal.
Source : Documentation officielle Git - git-gui

Question 22 : Pourquoi avez-vous choisi ce logiciel (Ungit) ?
Nous avons choisi **Ungit** pour son approche extrêmement visuelle. Il représente l'historique de Git sous forme de nœuds interactifs et d'animations très intuitives.
Source : Dépôt officiel GitHub de Ungit

Question 23 : Comment l'avez-vous installé (Ungit) ?
Ungit est un paquet Node.js. Après avoir installé Node.js et `npm` sur le système, nous l'avons installé via la commande globale : `sudo -H npm install -g ungit`. Ensuite, il se lance en tapant `ungit` et s'ouvre directement sous forme d'onglet dans le navigateur web.

Question 24 : Comparaison avec les outils inclus dans Git et la ligne de commande pure :
Par rapport à `gitk` ou `git-gui`, l'interface d'Ungit est bien plus moderne et unifie tout (historique, commit, push, merge) au même endroit de manière fluide. Par rapport à la ligne de commande pure, Ungit aide à mieux visualiser la topologie du projet, mais la ligne de commande reste plus rapide pour les opérations basiques et indispensable pour des commandes Git complexes.

Question 25 : Analyse comparative approfondie (Opérations)

- **Visualiser l'historique des commits :** `git log` ou `git log --oneline --graph` en CLI. Sur Ungit, il suffit d'ouvrir la page web et de scroller pour voir l'arbre interactif des nœuds colorés.
- **Créer une nouvelle branche :** `git branch <nom>` ou `git switch -c <nom>` en CLI. Sur Ungit, cliquer sur le nœud du commit actuel, taper le nom de la branche dans le champ de texte et cliquer sur le bouton "+ Branch".
- **Comparer deux versions d'un fichier (diff) :** `git diff` en CLI. (Visuel disponible sur Ungit).
- **Résoudre un conflit de fusion :** Ouvrir le fichier dans un éditeur (nano/vim), effacer les balises, garder le bon code, puis faire `git add` et `git commit` en CLI. Ungit signale le conflit en rouge ("Resolve conflicts"). On peut ouvrir un outil de diff visuel ou marquer le fichier comme résolu directement depuis l'interface après modification.

Question 26 : Quelles opérations sont plus rapides en ligne de commande ? Lesquelles sont plus claires avec l'interface graphique ?
Les opérations courantes, basiques et répétitives (comme le `git add`, la création de branche, un `git commit` simple ou un `git push`) sont beaucoup plus rapides à taper en ligne de commande. + En revanche, comprendre l'arborescence des branches, visualiser un historique complexe et repérer les modifications dans plusieurs fichiers en même temps est bien plus clair avec l'interface graphique.

Question 27 : Dans un contexte professionnel, quel outil privilégieriez-vous et pourquoi ?
Dans un contexte professionnel, l'idéal est de combiner les deux. La ligne de commande est privilégiée pour la rapidité des actions quotidiennes et pour l'automatisation de scripts. L'interface graphique (Ungit, GitKraken, ou un outil intégré à l'IDE comme VS Code) est en revanche indispensable pour faire des revues de code, comprendre l'historique d'un projet avant une fusion (merge), et résoudre des conflits complexes avec moins de risque d'erreur.

4. Semaine 11 : Questions et Réponses techniques (Forgejo)

Auteurs : Lefebvre Romain, Plantard Kylian, Belot Emilien

Navigation rapide : Pour voir notre déploiement de Forgejo et nos retours d'expérience, veuillez ouvrir le fichier **SAE203_Tutoriel.pdf** au chapitre **Semaine 11**.

Question 1 : Qu'est-ce que Forgejo ?
C'est une forge logicielle libre et auto-hébergée, permettant de gérer des dépôts Git, le suivi des bugs (issues), et l'intégration continue.
Source : FAQ officielle de Forgejo

Question 2 : Qu'est-ce qu'un fork ?
En développement logiciel, un "fork" (ou embranchement) consiste à copier le code source d'un logiciel existant pour créer un nouveau projet indépendant, souvent suite à un désaccord technique ou éthique avec l'équipe d'origine.
Source : Wikipédia - Fork (développement logiciel)

Question 3 : Retracez l'histoire de Forgejo. Qu'est-ce qui différencie sa gouvernance de celle de Gitea ?
L'histoire commence avec **Gogs**, une forge très légère écrite en Go. Gogs évoluant trop lentement au goût de certains contributeurs, ils ont forké le projet pour créer **Gitea**. Plus récemment, l'entreprise derrière Gitea a pris un virage commercial (création d'une entreprise à but lucratif, Gitea Ltd). En réaction, la communauté a forké Gitea pour créer **Forgejo**. La grande différence réside dans le fait que Gitea est désormais contrôlé par une entreprise commerciale, tandis que Forgejo garantit une gouvernance 100% communautaire et à but non lucratif, empêchant toute privatisation du code.

Question 4 : Par défaut, Forgejo écoute sur le port 3000. Pourquoi pas le port 80 (port standard HTTP) ?
Sous Linux, les ports inférieurs à 1024 (appelés ports privilégiés) nécessitent les droits d'administrateur (`root`) pour pouvoir être ouverts et écoutés. Faire tourner une application web complexe comme Forgejo avec les droits `root` est une faille de sécurité majeure. Forgejo tourne donc sous un utilisateur standard et utilise un port non privilégié comme le 3000. En production, on place généralement un "Reverse Proxy" (comme Nginx) qui écoute le port 80 et redirige le trafic.
Source : W3C - Architecture des ports privilégiés sous Linux

Question 5 : Pourquoi SQLite plutôt que PostgreSQL ou MySQL/MariaDB ?
SQLite est une base de données orientée fichier, ultra-légère et ne nécessitant aucune configuration de serveur (pas de service supplémentaire à faire tourner ou à sécuriser).
Source : Documentation officielle SQLite

Question 6 : Où se trouve le fichier de base de données SQLite ? Qui en est le propriétaire ?
Le fichier se trouve dans le répertoire de travail de l'application `/var/lib/forgejo/data/forgejo.db`. Le propriétaire de ce fichier est l'utilisateur système qui exécute le service Forgejo (`git`), afin qu'il puisse y lire et y écrire les données.

Question 7 : Qu'est-ce qu'un service systemd ?
C'est le gestionnaire de système et de services de Linux. Il permet de démarrer, arrêter et surveiller les processus fonctionnant en arrière-plan, et de s'assurer qu'ils redémarrent en cas de crash.
Source : Wiki Debian - systemd

Question 8 : Expliquez le fichier de service systemd.
Le fichier se divise en trois parties :

- `[Unit]` qui contient les métadonnées et les dépendances,
- `[Service]` qui définit comment exécuter l'application (l'utilisateur cible `User=git`, le chemin vers le binaire `ExecStart=…`, etc.)
- `[Install]` qui définit à quel moment de la séquence de démarrage le service doit être lancé.

Question 9 : Différence entre start, enable et restart ? Comment visualiser les logs ?

- `start` lance le service immédiatement
- `enable` configure le service pour qu'il se lance automatiquement au démarrage de la machine
- `restart` arrête puis redémarre le service. Pour visualiser les logs, on utilise la commande `sudo journalctl -u forgejo -f`.

Question 10 : Examinez les permissions de /etc/forgejo et app.ini. Pourquoi sont-elles importantes ?
Le fichier `app.ini` contient des mots de passe en clair (clés secrètes, mail, configuration interne). Ses permissions doivent être très restrictives (généralement `0640` ou `0600`) pour que seul l'utilisateur propriétaire (`forgejo` et `root`) puisse le lire.
Source : Wiki Debian - Permissions des fichiers

Question 11 : Le serveur SSH de Forgejo écoute sur quel port ? Mesures pour un accès Internet ?
Contrairement à d'autres méthodes de déploiement, l'installation recommandée de Forgejo sur Debian suggère d'utiliser le serveur SSH natif du système (OpenSSH), qui écoute sur le port standard `22`. (Dans notre configuration VirtualBox isolée, nous y accédons via une redirection sur le port `2222`).
Pour exposer le serveur sur Internet de manière sécurisée, il faudrait:

1. Configurer le HTTPS (certificats SSL/TLS via Let's Encrypt).
2. Activer un pare-feu (`ufw`) pour ne laisser passer que les ports nécessaires (80, 443, 22, 2222).
3. Installer `Fail2ban` pour bloquer les tentatives d'attaques par force brute.

Question 12 : Projet privé vs public ? Ajouter des équipiers, lier une issue à un commit, tester clone/push en SSH.
Un projet public est lisible (et clonable) par n'importe quel visiteur, même non authentifié. Un projet privé est invisible et inaccessible à moins d'être explicitement invité. Forgejo permet de fermer automatiquement un ticket lors d'un push en utilisant des mots-clés spécifiques dans le message du commit (ex: `Fixes #1`). Renseigner sa clé publique permet de cloner et de pousser en SSH sans taper son mot de passe.
Source : Documentation Git - Génération des clés SSH

Question 13 : Qu'est-ce que CI/CD ? Workflow simple ? Avantages en équipe ?
CI (Continuous Integration) : Automatisation des tests et de la vérification du code à chaque push d'un développeur, garantissant que le nouveau code ne casse pas l'application. CD (Continuous Delivery/Deployment) : Déploiement automatisé de l'application une fois les tests validés. Les avantages sont la réduction massive des bugs, des retours immédiats, la standardisation de la compilation et un gain de temps.
Source : GitLab - Guide sur la CI/CD

[1] Le mode intelligent est un serveur http qui comprend l'authentification et les push, tandis que le mode idiot est juste un serveur http basique qui distribue le dossier `.git`.